```
%%writefile vectadd.cu

#include <stdio.h>
#include <cuda_runtime.h>
// Vector Addition Kernel
__global__ void vector(float* A, float* B, float* C, int N)
{    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if(i < N)
  { C[i]=A[i]+B[i];}
}
int main()
{
    int N = 4;
    size_t size = N * sizeof(float);
    float A[] = {1,2,3,4};
    float B[] = {5,6,7,8};
    float C[4];
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);
    cudaMemcpy(d_A, A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, B, size, cudaMemcpyHostToDevice);
    dim3 blocks(N, N);
    dim3 threads(1,1);
    vector<<<blocks, threads>>>(d_A, d_B, d_C, N);
    cudaMemcpy(C, d_C, size, cudaMemcpyDeviceToHost);
    printf("Vector Addition:- \n");
    printf("Result Matrix:\n");
    for(int i = 0; i < N; i++)
    {printf("%f ", C[i]); }
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    return 0; }
```

```
Writing vectadd.cu
```

```
!nvcc vectadd.cu -o vectadd
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future rele
```

```
!./vectadd
```

```
Vector Addition:-
Result Matrix:
6.000000 8.000000 10.000000 12.000000
```